CiudadanoDigital AI

Planeación del proyecto · 10 sesiones de 1.5 horas (15 horas en total) · Proceso de construcción-aplicación

Institución: _____ **Docente:** _____ **Fecha de inicio:** _____

ACCIÓN GENERAL

Diseñar, construir y probar una versión mínima funcional (MVP) de **CiudadanoDigital AI**: una app móvil en Flutter conectada a una API en FastAPI que usa inteligencia artificial para detectar phishing y estafas en SMS, WhatsApp y enlaces, y para revisar si una red Wi-Fi es segura. Está dirigida a los ciudadanos de Medellín.

ACCIÓN	NÚMERO DE SESIONES 10 sesiones de 1.5 horas (proceso de construcción-aplicación)	RESPONSABLES
Arranque y configuración Definir el alcance del MVP, instalar el entorno (Python, Flutter SDK, Docker, Git y Claude Code), crear el repositorio y generar la estructura base del proyecto. **Entregable:** Repositorio con las carpetas `backend/` y `mobile_app/`.	**Sesión 1** 1.5 h · acumulado 1.5 de 15 h	Líder de proyecto + todo el equipo
Backend base FastAPI, configuración (`core/`), modelos SQLAlchemy de reportes ciudadanos, esquemas Pydantic, PostgreSQL con Docker Compose y migraciones con Alembic. **Entregable:** API funcionando en `/docs` y conectada a la base de datos.	**Sesión 2** 1.5 h · acumulado 3 de 15 h	Dev Backend / IA
Motor de IA/NLP y `POST /api/v1/scan-text` Reglas para detectar enlaces acortados, dominios falsos, mensajes con urgencia y suplantación de Bancolombia, Nequi, Daviplata o la DIAN, más un modelo ligero o una API de IA. La respuesta dice si es seguro, el porcentaje de riesgo y una explicación. **Entregable:** Endpoint probado con 20 mensajes reales o simulados.	**Sesión 3** 1.5 h · acumulado 4.5 de 15 h	Dev Backend / IA
`GET /api/v1/wifi-check` y pruebas Verificación de cifrado WPA2/WPA3, endpoint de reportes ciudadanos y pruebas automáticas con pytest. **Entregable:** Backend completo y probado.	**Sesión 4** 1.5 h · acumulado 6 de 15 h **HITO 1**	Dev Backend / IA + QA

ACCIÓN	NÚMERO DE SESIONES 10 sesiones de 1.5 horas (proceso de construcción-aplicación)	RESPONSABLES
App Flutter base Tema Material 3 (azul ciberseguridad, blanco y verde éxito), navegación y Dashboard con el botón «Escanear Enlace o Mensaje» y el indicador «Estado del Perfil: Protegido». **Entregable:** App que abre y navega entre pantallas.	**Sesión 5** 1.5 h · acumulado 7.5 de 15 h	Dev Móvil (Flutter)
Pantalla Escáner Campo de texto, servicio HTTP, modelos de datos y resultado interactivo (riesgo y explicación) con mensajes de carga y de error. **Entregable:** Escáner conectado a `/scan-text`.	**Sesión 6** 1.5 h · acumulado 9 de 15 h	Dev Móvil (Flutter)
Pantalla Alertas / Wi-Fi Lectura de la red actual (con los permisos de Android/iOS) y consulta a `/wifi-check`. **Entregable:** Las 3 pantallas funcionando.	**Sesión 7** 1.5 h · acumulado 10.5 de 15 h **HITO 2**	Dev Móvil (Flutter)
Integración completa App conectada a la API con Docker Compose, pruebas con casos reales de estafas y corrección de errores. **Entregable:** Flujo completo entre la app y la API sin errores críticos.	**Sesión 8** 1.5 h · acumulado 12 de 15 h	Todo el equipo (lo dirige QA)
Despliegue y documentación Publicar la API en la nube (Render, AWS o GCP), escribir los README (instalación, migraciones y ejecución) y, si sobra tiempo, agregar Redis y pgvector. **Entregable:** API publicada y README completos.	**Sesión 9** 1.5 h · acumulado 13.5 de 15 h	Dev Backend / IA + QA y Documentación
Cierre y presentación Pruebas finales, APK de demostración y presentación del producto. **Entregable:** MVP final presentado.	**Sesión 10** 1.5 h · acumulado 15 de 15 h **HITO 3**	Todo el equipo

Las filas en verde son los hitos de control. Las tareas que no se terminen en la sesión se completan de forma asíncrona antes de la sesión siguiente.

CLÁUSULAS DE CUMPLIMIENTO		
Define categorías (3)	**Escribe el proceso de control a la categoría**	**Escribe la consecuencia frente al incumplimiento**
1. Tiempo y asistencia Cumplir el cronograma de las 10 sesiones.	• Lista de asistencia en cada sesión. • Tablero de tareas (GitHub Projects o Trello) con el entregable de cada sesión. • Revisión del avance en los últimos 10 minutos de cada sesión.	• **1.ª vez:** llamado de atención y entregar la tarea pendiente antes de la siguiente sesión. • **2.ª vez:** se le reasignan las tareas y queda registrado. • **3.ª vez:** baja su nota de coevaluación y se informa al docente.
2. Calidad técnica de los entregables Código que funciona, está probado y está en el repositorio.	• Commits en GitHub al final de cada sesión. • Cada cambio entra por Pull Request y lo revisa otro integrante. • Revisión en los hitos (sesiones 4, 7 y 10): endpoints en Swagger, pruebas con pytest y app corriendo en el emulador.	• El cambio no entra a `main` hasta que se corrija; plazo máximo de 1 sesión. • Si no se corrige, la funcionalidad queda pendiente (se reduce el alcance) y se refleja en la coevaluación.
3. Comunicación, documentación y uso responsable de la IA y los datos Trabajo trazable, documentado y seguro.	• Bitácora corta por sesión (qué se hizo, qué falta, qué bloqueó). • README actualizado con cada funcionalidad. • Respuesta en el canal del equipo en menos de 24 horas. • Cada integrante debe entender y poder explicar el código que genere Claude Code. • No subir claves (`.env` va en `.gitignore`) y usar datos de prueba anónimos (Ley 1581 de 2012).	• Una tarea sin documentar no cuenta como terminada. • Si se suben claves o datos personales: se borran, se cambian las claves de inmediato y el responsable presenta la corrección en la siguiente sesión. • Si la falta de comunicación se repite, se lleva al líder o al docente.

Roles del equipo

ROL	RESPONSABILIDADES	NOMBRE DEL INTEGRANTE
Líder de proyecto	Coordina el cronograma, lleva la lista de asistencia y el tablero de tareas, dirige la revisión de cierre de cada sesión y registra los incumplimientos.	
Dev Backend / IA	Construye la API en FastAPI, la base de datos, el motor de detección de phishing, los endpoints y el despliegue.	
Dev Móvil (Flutter)	Construye la app, sus tres pantallas, el consumo de la API y los permisos del dispositivo.	
QA y Documentación	Hace las pruebas (pytest y manuales), revisa los Pull Requests, lleva la bitácora y los README y verifica que no se suban claves.	

Si el equipo es más pequeño, una persona puede asumir varios roles; se escribe su nombre en cada uno.

Hitos de control

HITO	SESIÓN	QUÉ SE ENTREGA	CÓMO SE VERIFICA
Hito 1	Sesión 4	Backend completo y probado	Endpoints probados en Swagger y pruebas con pytest en verde.
Hito 2	Sesión 7	Las 3 pantallas de la app funcionando	App corriendo en el emulador: Dashboard, Escáner y Alertas / Wi-Fi.
Hito 3	Sesión 10	MVP final presentado	Demo completa app + API, APK de demostración y README entregados.

Ajustes recomendados antes de empezar

1. **El tiempo es justo para todo lo pedido.**
   Se prioriza lo esencial: FastAPI, PostgreSQL, Flutter y la IA. Redis, pgvector y el despliegue en la nube quedan como opcionales para la sesión 9.

2. **El Wi-Fi no se puede revisar solo con el nombre de la red.**
   El backend no puede saber el cifrado de una red solo con su nombre. La app debe enviar el nombre y el tipo de seguridad: en Android se obtiene con permiso de ubicación; en iOS el usuario lo elige a mano. El endpoint recibirá esos dos datos.

3. **Corrección en la estructura del proyecto.**
   En Flutter el archivo de inicio es `lib/main.dart`, no `main.py`.

4. **Un prompt por sesión.**
   El prompt maestro se usa en la sesión 1 solo para crear la estructura. Después se usa un prompt por sesión (backend, IA, cada pantalla) y se prueba cada parte antes de seguir.